Fake News detection project

By: Marius Gören & Aitor Vergara

Goal

Find the best model to predict Fake News in another dataset

Data Preprocessing

Common processes were done to clean the data:

- Tokenization
- Lowercasing & Punctuation
- Removing Stopwords

```python
import re

def clean_text(text):

    # Remove special characters
    text = re.sub(r'[^a-zA-Z0-9\s]', ' ', str(text))

    # Remove single characters
    text = re.sub(r'\b[a-zA-Z]\b', '', str(text))

    # Remove multiple spaces
    text = re.sub(r'\s+', ' ', str(text)).strip()

    # Lowercase
    text = text.lower()

    return text
```

Random Forest optimized 95%

Accuracy was the same but more balanced with CountVectorizer

```
Confusion Matrix:
 [[3328  187]
 [ 150 3166]]
```

```
Confusion Matrix:
 [[3265  250]
 [  91 3225]]
```

	precision	recall	f1-score	support
0	0.973	0.929	0.950	3515
1	0.928	0.973	0.950	3316
accuracy			0.950	6831
macro avg	0.950	0.951	0.950	6831
weighted avg	0.951	0.950	0.950	6831

```
tfidf_lr = Pipeline([
    ("vec", TfidfVectorizer(
        lowercase=True,
        ngram_range=(1,2),
        min_df=2,
        max_df=0.9,
        max_features=100000,
        sublinear_tf=True
    )),
    ("rfc", RandomForestClassifier(
        n_estimators=500,
        max_depth=None,
        max_features='log2',
        min_samples_split=2,
        # class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])

tfidf_lr.fit(X_train, y_train)
```

SVM (Support Vector Machine) - LinearSVC

- No difference between CountVectorizer/TF-IDF
- Better with uncleaned data
- Slightly higher than RandomForestClassifier

```
Confusion Matrix:
 [[3349  166]
 [ 149 3167]]
```

```
Accuracy: 0.9538866930171278
              precision    recall  f1-score   support

           0      0.957     0.953     0.955      3515
           1      0.950     0.955     0.953      3316

    accuracy                          0.954      6831
   macro avg      0.954     0.954     0.954      6831
weighted avg      0.954     0.954     0.954      6831
```

```
tfidf_lr = Pipeline([
    ("vec", TfidfVectorizer(
        lowercase=True,
        ngram_range=(1,2),
        min_df=2,
        max_df=0.9,
        sublinear_tf=True
    )),
    ("clf", LinearSVC(
    ))
])

tfidf_lr.fit(X_train, y_train)
```

Hyperparameter tuning (GridSearchCV)

GridSearchCV used to find the best parameters for each classifier.

Also used for the vectorizer TF-IDF

```
#Best parameters after Grid Search

# LogisticRegression(C=10, max_iter=1000)
# Best RF parameters: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
# Best NB parameters: {'alpha': 0.5}
```

```
#Best parameters for vectorizer after Grid Search

# max_df: 0.9, max_features: 100000, min_df: 2, ngram_range: (1, 2)
```

```
from sklearn.model_selection import GridSearchCV

# Base model
rf = RandomForestClassifier(random_state=42)

# Hyperparameters
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

# Grid Search
grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=5,                # cross validation
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train_tf2, y_train)

# Best model found
best_rf = grid_search.best_estimator_

print("Best RF parameters:", grid_search.best_params_)

# Final evaluation
rf_best_pred = best_rf.predict(X_test_tf2)
```

✓ 1m 47.4s

```
Best RF parameters: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
```

Classifier comparison (SVM, Logistic, MultinomialNB, RF)

```
================================================
TF-IDF Config: {'ngram': (1, 1)}
================================================
TF-IDF Config: {'ngram': (1, 2)}
================================================
TF-IDF Config: {'ngram': (1, 2), 'max_features': 5000}
================================================
TF-IDF Config: {'ngram': (1, 2), 'max_df': 0.8, 'min_df': 5}
================================================
TF-IDF Config: {'ngram': (1, 2), 'max_df': 0.9, 'min_df': 2, 'max_features': 100000}
```

Cleaned Data

FINAL RESULTS:

	model	ngram	max_df	min_df	max_features	train_acc	\ test_acc	train_f1	test_f1	gap
3	LogisticRegression	(1, 2)	1.0	1	NaN	0.998573	0.942029	0.998573	0.942033	0.056544
12	LogisticRegression	(1, 2)	0.9	2	100000.0	0.995718	0.941883	0.995718	0.941890	0.053835
5	MultinomialNB	(1, 2)	1.0	1	NaN	0.990008	0.938369	0.990007	0.938332	0.051638
14	MultinomialNB	(1, 2)	0.9	2	100000.0	0.965741	0.937784	0.965739	0.937771	0.027957
9	LogisticRegression	(1, 2)	0.8	5	NaN	0.984664	0.937491	0.984665	0.937499	0.047173
0	LogisticRegression	(1, 1)	1.0	1	NaN	0.980638	0.933538	0.980639	0.933549	0.047099
6	LogisticRegression	(1, 2)	1.0	1	5000.0	0.969401	0.933099	0.969404	0.933110	0.036302
11	MultinomialNB	(1, 2)	0.8	5	NaN	0.946268	0.931782	0.946260	0.931761	0.014487
8	MultinomialNB	(1, 2)	1.0	1	5000.0	0.936276	0.926804	0.936260	0.926770	0.009472
2	MultinomialNB	(1, 1)	1.0	1	NaN	0.951686	0.926072	0.951673	0.926032	0.025613
1	RandomForest	(1, 1)	1.0	1	NaN	0.999854	0.922559	0.999854	0.922557	0.077295
13	RandomForest	(1, 2)	0.9	2	100000.0	0.999927	0.920217	0.999927	0.920232	0.079710
10	RandomForest	(1, 2)	0.8	5	NaN	0.999780	0.916118	0.999780	0.916125	0.083663
7	RandomForest	(1, 2)	1.0	1	5000.0	0.999597	0.913336	0.999597	0.913342	0.086261
4	RandomForest	(1, 2)	1.0	1	NaN	1.000000	0.906895	1.000000	0.906913	0.093105

Classifier comparison (SVM, Logistic, MultinomialNB, RF)

Uncleaned Data

FINAL RESULTS:

	model	ngram	max_df	min_df	max_features	train_acc	\ test_acc	train_f1	test_f1	gap
3	LogisticRegression	(1, 2)	1.0	1	NaN	0.999597	0.952276	0.999597	0.952276	0.047321
12	LogisticRegression	(1, 2)	0.9	2	100000.0	0.997621	0.950959	0.997621	0.950962	0.046662
0	LogisticRegression	(1, 1)	1.0	1	NaN	0.990410	0.948470	0.990411	0.948476	0.041940
9	LogisticRegression	(1, 2)	0.8	5	NaN	0.991911	0.947592	0.991911	0.947598	0.044319
14	MultinomialNB	(1, 2)	0.9	2	100000.0	0.970572	0.946714	0.970570	0.946699	0.023859
6	LogisticRegression	(1, 2)	1.0	1	5000.0	0.978588	0.945103	0.978589	0.945107	0.033485
5	MultinomialNB	(1, 2)	1.0	1	NaN	0.988690	0.943932	0.988689	0.943880	0.044758
11	MultinomialNB	(1, 2)	0.8	5	NaN	0.953918	0.942175	0.953913	0.942155	0.011743
2	MultinomialNB	(1, 1)	1.0	1	NaN	0.958054	0.938808	0.958040	0.938751	0.019246
1	RandomForest	(1, 1)	1.0	1	NaN	0.999927	0.938662	0.999927	0.938669	0.061265
13	RandomForest	(1, 2)	0.9	2	100000.0	1.000000	0.937344	1.000000	0.937353	0.062656
10	RandomForest	(1, 2)	0.8	5	NaN	0.999890	0.933392	0.999890	0.933406	0.066498
8	MultinomialNB	(1, 2)	1.0	1	5000.0	0.940339	0.932660	0.940323	0.932630	0.007679
7	RandomForest	(1, 2)	1.0	1	5000.0	0.999707	0.926804	0.999707	0.926814	0.072903
4	RandomForest	(1, 2)	1.0	1	NaN	1.000000	0.917582	1.000000	0.917576	0.082418

Accuracy improves almost 1% with no clean data

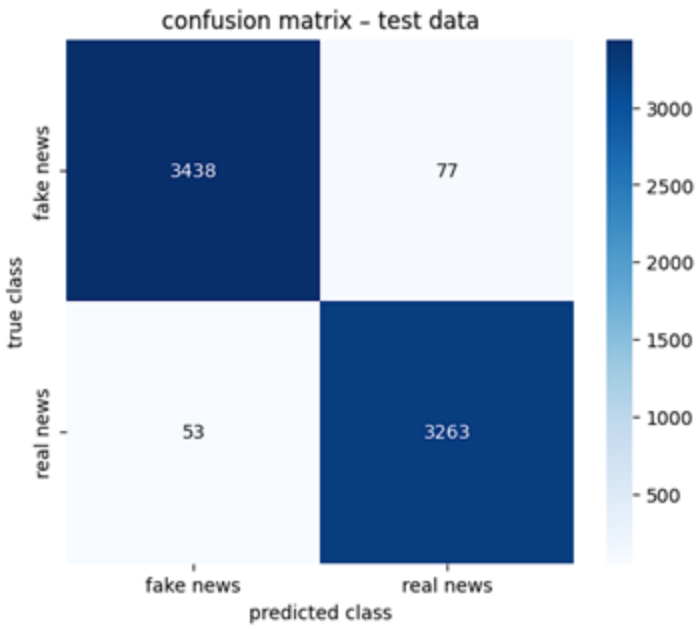
training pipeline of DistilBERT model (“distilbert-base-uncased”)

- Data preparation and splitting
  - Dataset is split into train, evaluation and test sets
  - better with uncleaned data
- Tokenization and Formatting
  - Text data is tokenized into numerical IDs with attention masks
    - then formatted into a HuggingFace Dataset.
- Training Configuration
  - TrainingArguments define hyperparameters like
    - batch size and learning rate to guide model optimization
- Model Training Execution
  - the Trainer class manages the training loop, batching, evaluation and checkpointing

performance result of DistilBERT model

- training duration:
  - 2.5 hours with a Laptop CPU
  - only 4 min with colab pro and T4 GPU
- high accuracy of **98.1 %**

	precision	recall	f1-score	support
0	0.985	0.978	0.981	3515.000
1	0.977	0.984	0.980	3316.000
accuracy	0.981	0.981	0.981	0.981
macro avg	0.981	0.981	0.981	6831.000
weighted avg	0.981	0.981	0.981	6831.000



lessons learnt

- difficult to gather information about “transformers”
    - Some sources are no longer up to date with the release of version 5.x
        - no longer works with tensorflow -> use PyTorch
        - parameters were renamed
- better with uncleaned data
    - this will keep important words and phrases to identify fake news
        - Recognizing writing style, emotion, patterns, rhetoric, trigger phrases and manipulative phrasing

Thank you